

AMAZON BRAZIL SALES & OPERATIONS REPORT

END-TO-END E-COMMERCE BUSINESS INTELLIGENCE USING MYSQL

By Vibha Pateshwari

PROJECT OBJECTIVE

Amazon Brazil is scaling rapidly across Latin America. The objective of this project is to analyze transactional order data across customers, sellers, products, payments, and reviews to extract actionable operational intelligence. This analysis empowers leadership to optimize regional fulfillment center investments, forecast seasonal spikes, audit catalog health, and design premium customer loyalty tiers.

PROBLEM STATEMENTS

1. Find the total number of orders fulfilled by each seller state.
2. For each product category, calculate the cumulative revenue generated over time.
3. Identify dominant customer payment methods and calculate Average Order Value (AOV).
4. Find the customer who has spent the most money across all orders.
5. Find the average review score for each product category.
6. Total orders placed per customer broken down by home state.
7. Identify dormant sellers with zero fulfilled orders.
8. Top 5 product categories by realized revenue.
9. Determine median delivery turnaround time in days.
10. Identify inactive product catalog listings with zero orders.
11. Sellers exceeding the platform average order fulfillment benchmark.
12. Brazilian states ranked by customer delivery review satisfaction.
13. Identify active buyers who have never submitted a product review.
14. Identify the historical peak order month across the marketplace.

1.FIND THE TOTAL NUMBER OF ORDERS FULFILLED BY EACH SELLER STATE.

```
• SELECT
    s.seller_state,
    COUNT(DISTINCT oi.order_id) AS total_orders
FROM sellers s
JOIN order_items oi ON s.seller_id = oi.seller_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status = 'delivered'
GROUP BY s.seller_state
ORDER BY total_orders DESC;
```

2.FOR EACH PRODUCT CATEGORY, CALCULATE THE CUMULATIVE REVENUE GENERATED OVER TIME.

- **SELECT**

```
p.product_category_name,  
o.order_purchase_timestamp,  
oi.price,  
SUM(oi.price) OVER (  
    PARTITION BY p.product_category_name  
    ORDER BY o.order_purchase_timestamp  
) AS cumulative_revenue
```

```
FROM order_items oi
```

```
JOIN orders o ON oi.order_id = o.order_id
```

```
JOIN products p ON oi.product_id = p.product_id
```

```
WHERE o.order_status = 'delivered';
```

3.IDENTIFY DOMINANT CUSTOMER PAYMENT METHODS AND CALCULATE AVERAGE ORDER VALUE (AOV).

- **SELECT**
 payment_type,
 COUNT(DISTINCT order_id) AS total_orders,
 ROUND(AVG(payment_value), 2) AS average_order_value
FROM order_payments
GROUP BY payment_type
ORDER BY total_orders DESC;

4.FIND THE CUSTOMER WHO HAS SPENT THE MOST MONEY ACROSS ALL THEIR ORDERS.

```
• SELECT
    c.customer_unique_id,
    ROUND(SUM(op.payment_value), 2) AS total_spent,
    COUNT(DISTINCT o.order_id) AS total_orders
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN order_payments op ON o.order_id = op.order_id
WHERE o.order_status = 'delivered'
GROUP BY c.customer_unique_id
ORDER BY total_spent DESC
LIMIT 1;
```

5.FIND THE AVERAGE REVIEW SCORE FOR EACH PRODUCT CATEGORY.

```
• SELECT
    COALESCE(t.product_category_name_english, p.product_category_name) AS category_name,
    ROUND(AVG(r.review_score), 2) AS avg_review_score,
    COUNT(r.review_id) AS total_reviews
FROM order_reviews r
JOIN order_items oi ON r.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
LEFT JOIN product_category_name_translation t
    ON p.product_category_name = t.product_category_name
WHERE p.product_category_name IS NOT NULL
GROUP BY category_name
ORDER BY avg_review_score ASC;
```

6.FIND THE TOTAL NUMBER OF ORDERS PLACED BY EACH CUSTOMER, BROKEN DOWN BY THE STATE THEY LIVE IN.

- **SELECT**
 c.customer_state,
 c.customer_unique_id,
 COUNT(o.order_id) **AS** total_orders
FROM customers c
JOIN orders o **ON** c.customer_id = o.customer_id
GROUP BY c.customer_state, c.customer_unique_id;

7.IDENTIFY SELLERS WHO REGISTERED ON THE PLATFORM BUT HAVE NEVER FULFILLED A SINGLE ORDER.

- ```
select s.seller_id
from sellers s
left join order_items oi on oi.seller_id = s.seller_id
left join orders o on oi.order_id = o.order_id
where o.order_id is NULL;
```

## 8.FIND THE TOP 5 PRODUCT CATEGORIES BY TOTAL REVENUE.

```
• SELECT
 p.product_category_name,
 ROUND(SUM(oi.price), 2) AS total_revenue
FROM products p
JOIN order_items oi ON p.product_id = oi.product_id
JOIN orders o ON oi.order_id = o.order_id
WHERE o.order_status = 'delivered'
GROUP BY p.product_category_name
ORDER BY total_revenue DESC
LIMIT 5;
```

## 9.FIND THE MEDIAN DELIVERY TIME (IN DAYS) BETWEEN ORDER PLACEMENT AND ACTUAL DELIVERY.

```
• WITH delivery_times AS (
 SELECT
 DATEDIFF(order_delivered_customer_date, order_purchase_timestamp) AS delivery_days,
 ROW_NUMBER() OVER (
 ORDER BY DATEDIFF(order_delivered_customer_date, order_purchase_timestamp)
) AS row_num,
 COUNT(*) OVER () AS total_count
 FROM orders
 WHERE order_status = 'delivered'
 AND order_delivered_customer_date IS NOT NULL
 AND order_purchase_timestamp IS NOT NULL
)
SELECT
 delivery_days AS median_delivery_days
FROM delivery_times
WHERE row_num = ROUND(total_count / 2);
```

## 10.FIND ALL PRODUCTS THAT HAVE NEVER BEEN ORDERED.

- `select p.product_id , oi.order_id  
from products p  
left join order_items oi  
on oi.product_id = p.product_id  
left join orders o on  
o.order_id= oi.order_id  
where oi.order_id is NULL;`

## 11.FIND SELLERS WHO HAVE FULFILLED MORE ORDERS THAN THE AVERAGE SELLER ON THE PLATFORM.

```
•with seller_order_counts as (
 select seller_id,
 count(distinct order_id) as total_orders
 from order_items
 group by seller_id
)

select
seller_id, total_orders
from seller_order_counts
where total_orders >
 (select avg(total_orders)
 from seller_order_counts);
```

## 12.FIND WHICH BRAZILIAN STATES HAVE THE HIGHEST AVERAGE CUSTOMER REVIEW SCORE FOR ORDERS DELIVERED THERE.

```
• select
 c.customer_state,
 avg(orn.review_score) as average_score
from customers c
join orders o
on o.customer_id = c.customer_id
join order_reviews orn
on orn.order_id = o.order_id
WHERE o.order_status = 'delivered'
group by c.customer_state
order by average_score desc
limit 1;
```

### 13.IDENTIFY CUSTOMERS WHO HAVE PLACED ORDERS BUT NEVER LEFT A REVIEW.

- ```
SELECT DISTINCT c.customer_id
FROM customers c
JOIN orders o
  ON c.customer_id = o.customer_id
WHERE NOT EXISTS (
  SELECT 1
  FROM order_reviews r
  WHERE r.order_id = o.order_id
);
```

14.FIND THE MONTH WITH THE HIGHEST NUMBER OF ORDERS PLACED ACROSS THE ENTIRE PLATFORM.

```
• select DATE_FORMAT(order_purchase_timestamp, '%Y-%m') as order_month,  
  count(order_id) as total_orders  
  from orders  
  group by order_month  
  order by total_orders desc  
  limit 1;
```

The background features a light gray field with a series of thin, white, wavy lines that create a sense of movement. In the corners, there are decorative elements: the top-left and bottom-right corners have thick, flowing bands of dark blue and gold, while the bottom-left corner contains a faint, white grid pattern.

THANK YOU